

# 정보처리기사\_필기\_이해중심\_개념지도

## 전체 지도

소프트웨어 시스템은 다음 흐름으로 이해하면 쉽습니다.

1. 무엇을 만들지 정한다 → 개발 방법론 · 요구사항
2. 어떤 구조로 만들지 그린다 → UML · 객체지향 · UI · 아키텍처
3. 코드와 데이터를 구성한다 → 자료구조 · 알고리즘 · 프로그래밍 언어 · 데이터베이스
4. 품질을 확인하고 배포한다 → 테스트 · 형상 관리 · 빌드 · 패키징 · 인터페이스
5. 컴퓨터와 네트워크에서 실행한다 → 운영체제 · 프로세스 · 메모리 · 통신 프로토콜
6. 프로젝트와 인프라를 운영한다 → 비용 · 일정 · 개발 표준 · 신기술
7. 시스템을 안전하게 지킨다 → 보안 개발 · 암호화 · 인증 · 공격 · 방어

## I. 무엇을 만들지 정하기

### 1. 개발의 큰 흐름

- 소프트웨어 공학
  - SE — Software Engineering (소프트웨어 공학)
  - 개발 전 과정을 나타내는 틀
    - SDLC — Software Development Life Cycle (소프트웨어 개발 생명주기)
    - 요구 분석 → 설계 → 구현 → 테스트 → 배포 → 유지보수
- SDLC를 진행하는 방식
  - 계획 중심 방식
    - 폭포수 모형 — Waterfall Model
      - 앞 단계가 끝난 뒤 다음 단계로 이동
    - 프로토타입 모형 — Prototype Model
      - 시제품을 먼저 만들어 요구사항 확인
    - 나선형 모형 — Spiral Model
      - 계획 → 위험 분석 → 개발·검증 → 고객 평가를 반복
  - 변화 대응 방식
    - Agile Model — 애자일 모형
      - 짧은 반복과 증분으로 개발

- Iteration — 반복
- Increment — 증분

## 2. 애자일이라는 큰 우산

- 애자일 방법론에 속하는 개발 방식

- Scrum — 스크럼
- XP — eXtreme Programming (익스트림 프로그래밍)
- Kanban — 칸반
- Lean — 린
- Crystal — 크리스탈
- ASD — Adaptive Software Development (적응형 소프트웨어 개발)
- FDD — Feature Driven Development (기능 중심 개발)
- DSDM — Dynamic Systems Development Method (동적 시스템 개발 방법론)
- DAD — Disciplined Agile Delivery (규율적 애자일 딜리버리)

- Scrum의 구성

- 사람
  - PO — Product Owner, 제품 책임자
  - SM — Scrum Master, 스크럼 마스터
  - DT — Development Team, 개발팀
- 작업 목록
  - Product Backlog — 제품 전체 작업
  - Sprint Backlog — 이번 스프린트 작업
- 반복 과정
  - Sprint — 스프린트
  - Daily Scrum — 일일 스크럼
  - Review — 검토
  - Retrospective — 회고
- 진행 상황 표현
  - Burndown Chart — 소멸 차트

- XP의 구성

- 핵심 가치
  - Communication (의사소통) · Simplicity (단순성) · Courage (용기) · Respect (존중) · Feedback (피드백)

- 협업 방식
  - Pair Programming — 짝 프로그래밍
  - Collective Ownership — 공동 코드 소유
  - Whole Team — 전체 팀
  - Planning Game — 계획 게임
- 코드 품질 방식
  - TDD — Test-Driven Development (테스트 주도 개발)
  - Refactoring — 리팩토링
  - Continuous Integration — 계속적 통합
- 배포 방식
  - Small Releases — 소규모 릴리즈

### 3. 요구사항: 만들 대상의 기준

- **요구사항의 출처**
  - User Requirement — 사용자 요구사항
  - System Requirement — 시스템 요구사항
- **요구사항의 성격**
  - Functional Requirement — 기능 요구사항
  - Non-functional Requirement — 비기능 요구사항
    - 성능 · 보안 · 신뢰성 · 사용성 · 유지보수성
- **요구사항에 포함되는 세부 영역**
  - 시스템 장비 구성 요구사항
  - 기능 요구사항
  - 인터페이스 요구사항
  - 데이터 요구사항
  - 테스트 요구사항
    - BMT — Benchmark Test (벤치마크 테스트)
  - 보안 요구사항
  - 품질 요구사항
  - 제약사항
- **요구공학의 흐름**
  1. Elicitation — 도출

- Interview (인터뷰) · Survey (설문) · Brainstorming (브레인스토밍) · Workshop (워크숍) · Prototype (프로토타입)
- 2. Analysis — 분석
  - 충돌·누락·중복 확인
- 3. Specification — 명세
  - Software Requirements Specification (소프트웨어 요구사항 명세서)
  - 정형 명세 / 비정형 명세
- 4. Validation — 확인
  - Review (리뷰) · Inspection (인스펙션) · Walkthrough (워크스루) · Prototype (프로토타입)
- 5. Management — 관리
  - Baseline (베이스라인) · Traceability (추적성) · Change Control (변경 통제)
- **현행 시스템을 파악할 때 보는 대상**
  - 업무 측면
    - 시스템 구성 · 기능 · 인터페이스
  - 실행 환경
    - OS — Operating System (운영체제)
    - DBMS — Database Management System (데이터베이스 관리 시스템)
      - Oracle · IBM DB2 · Microsoft SQL Server · MySQL · SQLite · MongoDB · Redis
    - WAS — Web Application Server (웹 애플리케이션 서버)
      - Tomcat · GlassFish · JBoss · Jetty · JEUS · Resin · WebLogic · WebSphere
  - 연결 환경
    - Network (네트워크) · Middleware (미들웨어)

## II. 구조를 설계하고 표현하기

### 1. 분석 모델: 요구사항을 구조로 바꾸는 도구

- 구조적 분석
  - DFD — Data Flow Diagram (자료 흐름도)
    - 데이터의 이동을 표현
  - DD — Data Dictionary (자료 사전)
    - DFD에 등장하는 데이터를 정리
  - Mini-Spec — Mini Specification (소단위 명세서)

- 처리 과정의 세부 로직을 정리
  - SADT — Structured Analysis and Design Technique (구조적 분석 및 설계 기법)
- **입력·처리·출력 중심 표현**
  - IPO — Input Process Output (입력·처리·출력)
  - HIPO — Hierarchy plus Input Process Output (계층적 입력·처리·출력)
- **요구사항 명세 기법**
  - 정형 명세 기법
    - 수학적 원리와 표기법 사용
    - VDM — Vienna Development Method (비엔나 개발 방법)
    - Z
    - Petri-net (페트리 넷)
    - CSP — Communicating Sequential Processes (통신 순차 프로세스)
  - 비정형 명세 기법
    - FSM — Finite State Machine (유한 상태 기계)
    - Decision Table (결정표)
    - ER Modeling (개체-관계 모델링)
    - State Chart (상태 차트)
- **분석·설계 자동화**
  - CASE — Computer-Aided Software Engineering (소프트웨어 공학 자동화)
    - Upper CASE — 분석·설계 지원
    - Lower CASE — 구현·테스트 지원
    - Integrated CASE — 전 과정 지원
  - PSL/PSA — Problem Statement Language / Problem Statement Analyzer
    - PSL (문제 기술 언어) · PSA (문제 기술 분석기)

## 2. UML: 시스템을 여러 관점으로 그리는 언어

- UML — Unified Modeling Language (통합 모델링 언어)
  - OMT — Object-Modeling Technique (객체 모델링 기법), Booch, Jacobson 방법론을 통합
  - OMG — Object Management Group (객체 관리 그룹)에서 표준화
  - 기본 재료
    - Thing — 사물
    - Relationship — 관계

- Diagram — 다이어그램
- Stereotype — 스테레오타입
- **정적인 구조를 보여주는 다이어그램**
  - Class Diagram — 클래스와 관계
  - Object Diagram — 특정 시점의 객체
  - Component Diagram — 구현 컴포넌트
  - Deployment Diagram — 물리적 배치
  - Package Diagram — 패키지 묶음
  - Composite Structure Diagram — 내부 구조
- **동적인 행동을 보여주는 다이어그램**
  - Use Case Diagram — 사용자와 기능
  - Sequence Diagram — 시간순 메시지
  - Communication Diagram — 객체 간 협력
  - State Diagram — 상태 변화
  - Activity Diagram — 작업 흐름
  - Timing Diagram — 시간 제약
  - Interaction Overview Diagram — 상호작용 전체 흐름
- **UML에서 객체를 연결하는 관계**
  - Association — 연관
    - Aggregation — 약한 포함
    - Composition — 강한 포함
  - Generalization — 일반화·상속
  - Dependency — 의존
  - Realization — 인터페이스 실체화

### 3. 객체지향: 프로그램을 객체의 협력으로 구성

- **객체를 만드는 구조**
  - Class — 설계도
    - Attribute — 상태
    - Method / Operation — 행동
  - Object / Instance — 클래스로 만든 실체
  - Message — 객체 사이의 요청

- **객체지향의 성질**

- Abstraction — 추상화
- Encapsulation — 캡슐화
  - Information Hiding — 정보 은닉
- Inheritance — 상속
- Polymorphism — 다형성
  - Overloading — 같은 이름, 다른 매개변수
  - Overriding — 상속받은 동작 재정의

- **객체지향 분석 방법론**

- Rumbaugh Method (럼바우 방법)
  - 객체 모델 · 동적 모델 · 기능 모델
- Booch Method (부치 방법)
- Jacobson Method (야콥슨 방법)
  - Use Case 중심

- **SOLID 설계 원칙**

- SRP — Single Responsibility Principle (단일 책임 원칙)
- OCP — Open-Closed Principle (개방-폐쇄 원칙)
- LSP — Liskov Substitution Principle (리스코프 치환 원칙)
- ISP — Interface Segregation Principle (인터페이스 분리 원칙)
- DIP — Dependency Inversion Principle (의존 역전 원칙)

- **계약으로 객체의 책임을 표현**

- DBC — Design by Contract (계약에 의한 설계)
  - Precondition — 선행 조건
  - Postcondition — 결과 조건
  - Invariant — 불변 조건

## 4. 사용자와 만나는 구조

- **UI — User Interface (사용자 인터페이스)**

- 입력 방식에 따른 분류
  - CLI — Command Line Interface (명령줄 인터페이스)
  - GUI — Graphical User Interface (그래픽 사용자 인터페이스)
  - NUI — Natural User Interface (자연 사용자 인터페이스)

- VUI — Voice User Interface (음성 사용자 인터페이스)
- OUI — Organic User Interface (유기적 사용자 인터페이스)
- 좋은 UI의 기본 원칙
  - Intuitiveness (직관성) · Learnability (학습성) · Effectiveness (유효성) · Flexibility (유연성)
- UI 설계 산출물
  - Wireframe (와이어프레임) → Mockup (목업) → Storyboard (스토리보드) → Prototype (프로토타입)
- **UI보다 넓은 사용자 경험**
  - UX — User Experience (사용자 경험)
  - Usability — 사용성
- **UI 화면을 구성하는 요소**
  - Input Control (입력 제어)
  - Navigation (내비게이션)
  - Output (출력)
  - Help (도움말)
- **소프트웨어 품질 특성 — ISO/IEC 9126**
  - Functionality (기능성)
  - Reliability (신뢰성)
  - Usability (사용성)
  - Efficiency (효율성)
  - Maintainability (유지보수성)
  - Portability (이식성)

## 5. 시스템 전체 구조

- **설계 수준**
  - High-level Design — 아키텍처·데이터·인터페이스
  - Low-level Design — 모듈·자료구조·알고리즘
- **아키텍처 패턴**
  - Layered Pattern — 계층별 책임 분리
  - Client-Server Pattern — 요청자와 제공자 분리
  - MVC — Model View Controller (모델-뷰-컨트롤러)
    - Model — 데이터·업무 규칙
    - View — 화면



- Controller — 입력·흐름 제어
- Pipe-Filter Pattern — 필터를 거치며 데이터 변환
- Repository Pattern — 중앙 저장소 공유
- Broker Pattern — 중개자를 통한 분산 호출
- **아키텍처 품질 속성**
  - 실행 중 확인되는 속성
    - Performance (성능) · Security (보안) · Availability (가용성) · Usability (사용성)
  - 개발·변경 과정에서 확인되는 속성
    - Modifiability (변경 용이성) · Testability (시험 용이성) · Maintainability (유지보수성)

## 6. 모듈을 잘 나누는 기준

- **좋은 모듈 구조**
  - Cohesion — 응집도는 높게
    - 기능적 → 순차적 → 교환적 → 절차적 → 시간적 → 논리적 → 우연적 응집도
  - Coupling — 결합도는 낮게
    - 자료 → 스탬프 → 제어 → 외부 → 공통 → 내용 결합도
- **모듈 호출 관계**
  - Fan-In (팬인) — 자신을 호출하는 상위 모듈 수
  - Fan-Out (팬아웃) — 자신이 호출하는 하위 모듈 수
- **공통 모듈 명세의 기준**
  - Accuracy (정확성)
  - Clarity (명확성)
  - Completeness (완전성)
  - Consistency (일관성)
  - Traceability (추적성)
- **코드를 부여하는 방식**
  - Sequence Code (순차 코드)
  - Block Code (블록 코드)
  - Decimal Code (10진 코드)
  - Group Classification Code (그룹 분류 코드)
  - Significant Digit Code (표의 숫자 코드)
  - Mnemonic Code (연상 코드)

- Combined Code (합성 코드)
- **제어 구조 표현**
  - Flowchart (순서도) · Pseudocode (의사코드)
  - N-S Chart — Nassi-Shneiderman Chart (나시-슈나이더만 차트)

## 7. 반복되는 설계 문제의 해법

- **GoF 디자인 패턴**
  - 객체를 만드는 방법 — 생성 패턴
    - Abstract Factory (추상 팩토리) · Builder (빌더) · Factory Method (팩토리 메소드) · Prototype (프로토타입) · Singleton (싱글톤)
  - 객체를 조립하는 방법 — 구조 패턴
    - Adapter (어댑터) · Bridge (브리지) · Composite (컴포지트) · Decorator (데코레이터) · Facade (퍼싸드) · Flyweight (플라이웨이트) · Proxy (프록시)
  - 객체가 협력하는 방법 — 행위 패턴
    - Chain of Responsibility (책임 연쇄) · Command (커맨드) · Interpreter (인터프리터) · Iterator (반복자)
    - Mediator (중재자) · Memento (메멘토) · Observer (옵서버) · State (상태) · Strategy (전략)
    - Template Method (템플릿 메소드) · Visitor (방문자)

## 8. 애플리케이션 사이를 연결하는 미들웨어

- **DB 기반 미들웨어**
  - DB Middleware (데이터베이스 미들웨어)
  - ODBC · JDBC
- **원격 프로시저 호출**
  - RPC — Remote Procedure Call (원격 프로시저 호출)
- **메시지 기반 통신**
  - MOM — Message-Oriented Middleware (메시지 지향 미들웨어)
- **객체 기반 통신**
  - ORB — Object Request Broker (객체 요청 브로커)
  - CORBA — Common Object Request Broker Architecture (공통 객체 요청 브로커 구조)
- **트랜잭션 처리**
  - TP Monitor — Transaction Processing Monitor (트랜잭션 처리 모니터)

# III. 코드와 품질을 관리하기

## 1. 기능을 구현 단위로 나누기

- 단위 모듈

- 한 가지 기능 또는 목적을 수행하며 독립적으로 컴파일할 수 있는 구현 단위
- 구현 순서
  - 단위 기능 명세서 작성 → 입·출력 기능 구현 → 알고리즘 구현

- 프로세스 사이의 통신

- IPC — Inter-Process Communication (프로세스 간 통신)
  - Shared Memory (공유 메모리) · Socket (소켓) · Semaphore (세마포어) · Pipe (파이프)

## 2. 데이터를 코드 안에서 담는 구조

- 자료구조

- 선형 구조

- Array — 배열
- Linear List / Linked List — 선형·연결 리스트
- Stack (스택) — LIFO 구조
  - Push (삽입) · Pop (삭제)
- Queue (큐) — FIFO 구조
  - Enqueue (삽입) · Dequeue (삭제) · Front (앞) · Rear (뒤)
- Deque — 양쪽 입출력

- 비선형 구조

- Tree (트리)
  - Binary Tree (이진 트리)
  - Preorder (전위 순회) · Inorder (중위 순회) · Postorder (후위 순회)
- Graph (그래프)
  - Directed Graph (방향 그래프) · Undirected Graph (무방향 그래프)

- 자료를 찾는 알고리즘

- Linear Search — 선형 검색
- Binary Search — 이분 검색
- Hashing — 해싱
  - Hash Function (해싱 함수)

- Division (제산법) · Folding (폴딩법) · Mid-Square (중간 제곱법)
- Collision (충돌) → Synonym (동의어) → Overflow (오버플로)
- 자료를 정렬하는 알고리즘
  - 단순 정렬
    - Insertion Sort (삽입 정렬) · Selection Sort (선택 정렬) · Bubble Sort (버블 정렬)
  - 분할·트리 기반 정렬
    - Quick Sort (퀵 정렬) · Merge Sort (합병 정렬) · Heap Sort (힙 정렬) · Shell Sort (셸 정렬)
  - 효율 표현
    - Time Complexity (시간 복잡도) · Space Complexity (공간 복잡도)
    - Big-O Notation (빅오 표기법)

### 3. 프로그래밍 언어의 공통 뼈대

- 데이터 표현
  - Primitive Type (기본 자료형) · Derived Type (파생 자료형)
  - Variable (변수) · Constant (상수) · Casting (형변환)
  - Garbage Collector (가비지 컬렉터)
- 연산
  - Arithmetic (산술) · Relational (관계) · Bitwise (비트) · Logical (논리) · Assignment (대입) · Conditional Operator (조건 연산자)
- 제어 구조
  - Sequence (순차)
  - Selection (선택) — if · switch
  - Iteration (반복) — for · while · do-while
  - 흐름 변경 — break · continue
- 언어별 연결 지도
  - C / C++
    - 메모리에 직접 접근 — Pointer
    - 동적 메모리 — malloc / free
    - 입출력 — scanf / printf
    - 주요 제어문자 — `\n` · `\t` · `\b` · `\r` · `\f` · `\0`
    - 배열의 선언과 초기화
    - 주요 헤더 — stdio.h · stdlib.h · string.h · math.h
  - Java

- 예외 처리 — try → catch → finally
- 객체 관계 — extends · implements
- Python
  - Sequence (시퀀스) — List (리스트) · Tuple (튜플) · String (문자열)
  - Mapping (매핑) — Dictionary (딕셔너리)
  - Set — 집합
  - 범위 추출 — Slice
  - 함수·클래스 — def · lambda · class
- **프로그래밍 패러다임별 대표 언어**
  - 절차형 언어
    - ALGOL (알골) · COBOL (코볼) · FORTRAN (포트란) · PASCAL (파스칼) · C
  - 객체지향 언어
    - JAVA (자바) · C++ · Smalltalk (스몰토크)
  - 스크립트 언어
    - JavaScript (자바스크립트) · VBScript (VB 스크립트)
    - ASP — Active Server Pages (액티브 서버 페이지)
    - JSP — JavaServer Pages (자바 서버 페이지)
    - PHP — Professional Hypertext Preprocessor
    - Python (파이썬) · Shell Script (셸 스크립트) · BASIC (베이직)
  - 선언형 언어
    - SGML — Standard Generalized Markup Language (표준 범용 문서 생성 언어)
      - HTML — HyperText Markup Language (하이퍼텍스트 마크업 언어)
      - XML — eXtensible Markup Language (확장성 마크업 언어)
    - LISP (리스프) · PROLOG (프롤로그) · Haskell (하스켈)
- **라이브러리**
  - Standard Library (표준 라이브러리)
  - External Library (외부 라이브러리)

## 4. 소스가 실행 파일이 되기까지

- **통합 개발 환경**
  - IDE — Integrated Development Environment (통합 개발 환경)
    - Editor (편집기) · Compiler (컴파일러) · Debugger (디버거) · Build Tool (빌드 도구)

- **빌드 도구**

- Make · Ant · Maven · Gradle
  - Gradle의 스크립트 — DSL, Domain Specific Language (도메인 특화 언어)
- 빌드 자동화 서버
  - Jenkins

- **형상 관리**

- SCM — Software Configuration Management (소프트웨어 형상 관리)
- 중앙집중형
  - SVN — Subversion (서브버전)
- 분산형
  - Git
- 그 밖의 형상 관리 도구
  - CVS — Concurrent Versions System (동시 버전 시스템)
  - RCS — Revision Control System (개정 제어 시스템)
  - SCCS — Source Code Control System (소스 코드 제어 시스템)
  - PVCS — Polytron Version Control System (폴리트론 버전 제어 시스템)
  - QVCS — Quma Version Control System (큐마 버전 제어 시스템)
  - CMVC — Configuration Management Version Control (형상 관리 버전 제어)
  - CVSNT · ClearCase · Perforce
  - DVC · GNU arch · Bazaar · Mercurial · TeamWare · BitKeeper · Plastic SCM
- 공통 작업
  - Repository (저장소) · Check-out (체크아웃) · Commit (커밋) · Branch (브랜치) · Merge (병합) · Tag (태그)
  - Configuration Item (형상 항목) · Baseline (베이스라인) · Version (버전) · Audit (감사)

- **배포할 수 있는 형태로 묶기**

- Packaging (패키징) → Release Note (릴리즈 노트) → Installation Manual (설치 매뉴얼) → Deployment (배포)
- 문제가 생기면 Patch 또는 Rollback

- **디지털 콘텐츠 권리 관리**

- DRM — Digital Rights Management (디지털 저작권 관리)
  - Content Provider (콘텐츠 제공자) → Packager (패키저) → Content Distributor (콘텐츠 분배자) → Consumer (사용자)
  - Clearing House — 권한·과금 관리

- Security Container — 암호화된 콘텐츠
- DRM Controller — 사용 권한 제어

## 5. 테스트를 목적과 단계로 연결하기

### • 코드를 실행하는가?

- Static Test — 실행하지 않음
  - Review (리뷰) · Walkthrough (워크스루) · Inspection (인스펙션)
- Dynamic Test — 실행함
  - White Box Test (화이트박스 테스트) · Black Box Test (블랙박스 테스트)

### • 내부 구조를 보는가?

- White Box Test — 내부 경로 기준
  - Base Path Testing (기초 경로 검사)
  - Control Structure Testing (제어 구조 검사)
  - Coverage (검증 기준)
    - Statement (문장) · Branch (분기) · Condition Coverage (조건 검증)
  - Cyclomatic Complexity — 순환 복잡도
- Black Box Test — 입력과 출력 기준
  - Equivalence Partitioning (동치 분할)
  - Boundary Value Analysis (경계값 분석)
  - Cause-Effect Graphing (원인-효과 그래프)
  - Decision Table (결정 테이블) · State Transition (상태 전이) · Error Guessing (오류 예측)

### • 어느 크기에서 검사하는가?

1. Unit Test — 모듈
2. Integration Test — 모듈 연결
  - Top-Down (하향식)
    - Test Stub 필요
  - Bottom-Up (상향식)
    - Test Driver 필요
3. System Test — 전체 시스템
4. Acceptance Test — 사용자 요구 충족
  - Alpha Test (알파 테스트) · Beta Test (베타 테스트)

### • 왜 검사하는가?

- Recovery (회복) · Security (안전) · Stress (강도) · Performance Test (성능 테스트)
- Regression Test — 변경 후 기존 기능 재검사
- **테스트 보조 구성**
  - Test Case (테스트 케이스) → Test Scenario (테스트 시나리오)
  - Test Oracle (테스트 오라클) — 예상 결과 판단
  - Test Harness (테스트 하네스) — 실행 환경
  - 자동화 도구
    - 단위 테스트 — JUnit · CppUnit
    - 웹 테스트 — Selenium · watir
    - 인수 테스트 — FitNesse
    - 통합 자동화 — NTAF · STAF
- **인터페이스 구현 검증 도구**
  - xUnit — JUnit · CppUnit 등
  - STAF — Software Testing Automation Framework (소프트웨어 테스트 자동화 프레임워크)
  - FitNesse · NTAF — NHN Test Automation Framework (NHN 테스트 자동화 프레임워크)
  - Selenium · watir — Web Application Testing in Ruby (루비 웹 애플리케이션 테스트)
- **데이터 무결성 검사 도구**
  - Tripwire · AIDE — Advanced Intrusion Detection Environment (고급 침입 탐지 환경)
  - Samhain · Claymore · Slipwire · Fcheck
- **결함의 관계**
  - Error — 사람이 만든 오류
  - Fault / Defect — 코드에 남은 결함
  - Failure — 실행 중 나타난 실패
  - 결함 상태
    - Open (등록) → Assigned (할당) → Fixed (수정) → Closed (종료)
    - Deferred (연기) · Classified (오류 아님)
- **성능 측정**
  - Throughput (처리량) · Response Time (응답 시간) · Turnaround Time (경과 시간) · Resource Usage (자원 사용률)
  - APM — Application Performance Management / Monitoring (애플리케이션 성능 관리·모니터링)
  - Scouter (스카우터)

## 6. 서로 다른 시스템 연결하기



- **단순 직접 연결**
    - DB Link (DB 링크) · DB Connection (DB 연결) · JDBC (자바 데이터베이스 연결) · API (응용 프로그래밍 인터페이스) · Socket (소켓)
  - **기업 내부 애플리케이션 통합**
    - EAI — Enterprise Application Integration (기업 애플리케이션 통합)
      - Point-to-Point (포인트 투 포인트)
      - Hub & Spoke (허브 앤 스포크)
      - Message Bus (메시지 버스)
      - Hybrid (하이브리드)
    - ESB — Enterprise Service Bus (기업 서비스 버스)
      - 서비스 버스를 중심으로 느슨하게 연결
  - **웹 서비스 방식**
    - SOAP 방식
      - SOAP — 메시지 규약
      - WSDL — 서비스 명세
      - UDDI — 서비스 검색
      - XML — 데이터 표현
    - REST 방식
      - HTTP 메소드 사용
      - JSON 또는 XML로 데이터 표현
    - 화면 비동기 통신
      - AJAX — Asynchronous JavaScript and XML (비동기 자바스크립트와 XML)
- 

## IV. 데이터를 설계하고 운영하기

### 1. 현실의 정보를 테이블로 바꾸는 과정

1. 요구조건 분석
2. 개념적 설계
  - 현실 개념을 ERD로 표현
  - Entity (개체) · Attribute (속성) · Relationship (관계)
  - 관계의 수 — 1:1 · 1:N · N:M
3. 논리적 설계

- 관계형 모델이라면 Relation으로 변환
- DBMS가 이해하는 논리 구조 결정

#### 4. 물리적 설계

- File · Record · Index · Partition · Storage 결정

#### 5. 구현

- DDL로 실제 객체 생성

## 2. 관계형 데이터베이스의 구조

### • 3단계 스키마

- External Schema (외부 스키마) — 사용자·응용 프로그램 관점
- Conceptual Schema (개념 스키마) — 데이터베이스 전체 관점
- Internal Schema (내부 스키마) — 물리적 저장 관점

### • Relation — 테이블

- Row (행)
  - Tuple (튜플)
  - 전체 개수 = Cardinality
- Column (열)
  - Attribute (속성)
  - 전체 개수 = Degree
- 값의 허용 범위
  - Domain (도메인)

### • 튜플을 식별하는 키

- Super Key — 유일성을 만족
  - Candidate Key — 유일성 + 최소성
    - Primary Key — 대표 후보키
    - Alternate Key — 선택되지 않은 후보키
- Foreign Key — 다른 릴레이션의 기본키 참조

### • 데이터가 지켜야 할 무결성

- Entity Integrity — 기본키는 NULL·중복 불가
- Referential Integrity — 외래키는 참조 가능한 값
- Domain Integrity — 속성은 도메인 범위 준수

## 3. 관계형 데이터를 계산하는 원리

- 관계대수 — 절차적 표현

- 한 릴레이션에 적용
  - SELECT  $\sigma$  — 행 선택
  - PROJECT  $\pi$  — 열 선택
- 여러 릴레이션에 적용
  - JOIN  $\bowtie$  — 관련 튜플 결합
  - DIVISION  $\div$  — 모두 만족하는 튜플
- 집합 연산
  - UNION (합집합)  $\cup$  · INTERSECTION (교집합)  $\cap$  · DIFFERENCE (차집합)  $-$  · PRODUCT (교차곱)  $\times$

- 관계해석 — 비절차적 표현

- Tuple Relational Calculus (튜플 관계해석)
- Domain Relational Calculus (도메인 관계해석)

## 4. 중복을 줄이는 설계: 정규화

- 정규화가 해결하는 문제

- Insertion Anomaly (삽입 이상) · Deletion Anomaly (삭제 이상) · Update Anomaly (갱신 이상)

- 함수적 종속을 먼저 파악

- Determinant (결정자)  $\rightarrow$  Dependent (종속자)
- FD — Functional Dependency (함수적 종속)
  - Full Functional Dependency (완전 함수적 종속)
  - Partial Functional Dependency (부분 함수적 종속)
  - Transitive Functional Dependency (이행 함수적 종속)

- 정규형은 앞 단계를 만족해야 다음 단계로 이동

- 1NF — 원자값
  - 2NF — 부분 함수 종속 제거
    - 3NF — 이행 함수 종속 제거
      - BCNF — 모든 결정자가 후보키
        - 4NF — 다치 종속 제거
          - 5NF — 조인 종속 제거

- 성능 때문에 정규화를 일부 되돌림

- Denormalization — 반정규화
  - Table Integration (테이블 통합) · Table Division (테이블 분할)

- Redundant Column (중복 컬럼) · Derived Column (파생 컬럼)
- Clustering (클러스터링) · Partitioning (파티셔닝)

## 5. SQL을 목적에 따라 연결하기

- 구조를 정의 — **DDL, Data Definition Language**
  - CREATE · ALTER · DROP
- 데이터를 다룸 — **DML, Data Manipulation Language**
  - 조회 — SELECT
    - 행 필터 — WHERE
    - 그룹화 — GROUP BY → HAVING
    - 정렬 — ORDER BY
    - 다른 질의 포함 — Subquery
    - 테이블 연결 — JOIN
      - INNER (내부) · LEFT/RIGHT/FULL OUTER (외부) · CROSS (교차) · SELF JOIN (자체 조인)
    - 질의 결과 결합
      - UNION · UNION ALL · INTERSECT · EXCEPT
  - 추가 — INSERT
  - 수정 — UPDATE
  - 삭제 — DELETE
- 데이터를 제어 — **DCL, Data Control Language (교제 분류)**
  - GRANT · REVOKE
  - COMMIT · ROLLBACK · SAVEPOINT
- **DB 안에서 실행되는 프로그램**
  - Stored Procedure (저장 프로시저) · User-Defined Function (사용자 정의 함수) · Trigger (트리거)
  - Embedded SQL (내장 SQL) · ODBC — Open Database Connectivity (개방형 데이터베이스 접속 규격)
- **DBMS를 관리하는 주체와 내부 정보**
  - DBA — Database Administrator (데이터베이스 관리자)
  - System Catalog (시스템 카탈로그)
    - DB 객체와 제약조건 정보를 저장하는 시스템 테이블
    - Data Dictionary (데이터 사전)라고도 함
- **객체와 관계형 DB를 연결**
  - ORM — Object-Relational Mapping (객체-관계 매핑)

- **DBMS 접속 기술**
  - JDBC — Java Database Connectivity
  - ODBC — Open Database Connectivity
  - MyBatis
- **데이터 변환 과정**
  - ETL — Extraction, Transformation, Loading (추출·변환·적재)
- **SQL 실행 계획을 선택하는 옵티마이저**
  - RBO — Rule-Based Optimizer (규칙 기반 옵티마이저)
  - CBO — Cost-Based Optimizer (비용 기반 옵티마이저)

## 6. 트랜잭션과 동시에 접근하는 데이터

- **Transaction (트랜잭션)**
  - 트랜잭션의 특성
    - Atomicity (원자성) · Consistency (일관성) · Isolation (독립성) · Durability (영속성)
  - 상태 흐름
    - Active (활동) → Partially Committed (부분 완료) → Committed (완료)
    - Active (활동) → Failed (실패) → Aborted (철회)
  - 데이터 작업 분류
    - CRUD — Create (생성) · Read (읽기) · Update (갱신) · Delete (삭제)
- **Concurrency Control — 병행 제어**
  - Locking (로킹)
    - 2PL — Two-Phase Locking (2단계 로킹)
  - Timestamp Ordering (타임스탬프 순서)
- **Recovery — 회복**
  - Log 기반
    - REDO (재실행) · UNDO (취소)
    - Deferred Update (연기 갱신) · Immediate Update (즉각 갱신)
  - Checkpoint (검사점)
  - Shadow Paging (그림자 페이징)
- **Deadlock — 교착상태**
  - 네 조건이 동시에 성립
    - Mutual Exclusion (상호 배제)

- Hold and Wait (점유와 대기)
- No Preemption (비선점)
- Circular Wait (환형 대기)

## 7. 성능과 저장 구조

- **검색 속도를 높임**
  - Index (인덱스)
    - B-Tree (B 트리) · B+ Tree (B+ 트리) · Bitmap (비트맵)
    - Clustered (클러스터드) · Non-clustered Index (비클러스터드 인덱스)
- **질의를 단순하게 보이게 함**
  - View (뷰)
  - Materialized View — 결과를 실제 저장
- **큰 테이블을 나눔**
  - Partition (파티션)
    - Range (범위) · Hash (해시) · List (목록) · Composite Partitioning (복합 파티셔닝)
  - Fragment (분할)
    - Horizontal (수평) · Vertical Fragmentation (수직 분할)
- **저장장치 연결 방식**
  - DAS — Direct Attached Storage (직접 연결 저장장치)
    - SATA — Serial Advanced Technology Attachment (직렬 ATA)
    - eSATA — external SATA (외장형 SATA)
    - SCSI — Small Computer System Interface (소형 컴퓨터 시스템 인터페이스)
    - SAS — Serial Attached SCSI (직렬 연결 SCSI)
  - NAS — Network Attached Storage (네트워크 연결 저장장치)
  - SAN — Storage Area Network (저장장치 전용 네트워크)
    - iSCSI — Internet Small Computer System Interface (인터넷 SCSI)
    - FC — Fibre Channel (파이버 채널)
    - FCoE — Fibre Channel over Ethernet (이더넷 기반 파이버 채널)
- **여러 장소에 DB를 배치**
  - Distributed Database (분산 데이터베이스)
    - Location (위치) · Replication (중복) · Failure Transparency (장애 투명성)

# V. 프로그램이 실행되고 통신하는 환경

## 1. 운영체제의 역할

- **OS — Operating System (운영체제)**
  - 핵심부 — Kernel
  - 사용자 명령 해석 — Shell
  - 처리 프로그램
    - Language Translator (언어 번역 프로그램) · Service Program (서비스 프로그램)
  - 제어 프로그램
    - Supervisor (감시 프로그램) · Job Control (작업 제어) · Data Management Program (데이터 관리 프로그램)
  - 대표 계열
    - DOS — Disk Operating System (디스크 운영체제)
    - Windows
    - UNIX / LINUX (유닉스 / 리눅스)
- **Windows의 주요 용어**
  - GUI — Graphical User Interface (그래픽 사용자 인터페이스)
  - PnP — Plug and Play (플러그 앤 플레이)
  - OLE — Object Linking and Embedding (객체 연결 및 삽입)
  - VFAT — Virtual File Allocation Table (가상 파일 할당 테이블)
- **UNIX 시스템의 구성**
  - Kernel (커널) → Shell (셸) → Utility (유틸리티)
  - File Descriptor (파일 디스크립터)
    - FCB — File Control Block (파일 제어 블록)
  - 주요 환경 변수
    - PATH · HOME · SHELL · PS1 · PWD · TERM
  - 주요 명령어
    - `cat` · `chmod` · `chown` · `cp` · `ls` · `rm` · `ps` · `kill` · `fork` · `exec`

## 2. 실행 단위와 CPU 배분

- **Program이 실행되면 Process**
  - 관리 정보 — PCB, Process Control Block
  - 더 작은 실행 단위 — Thread (스레드)

- 프로세스 전환 — Context Switching
- 상태
  - Ready (준비) → Running (실행) → Waiting/Blocked (대기) → Ready (준비)
  - Running (실행) → Terminated (종료)
- **CPU Scheduling (CPU 스케줄링)**
  - 비선점 방식
    - FCFS — First Come First Served (선입 선처리)
    - SJF — Shortest Job First (최단 작업 우선)
    - HRN — Highest Response-ratio Next (최고 응답률 우선)
    - Non-preemptive Priority (비선점 우선순위)
  - 선점 방식
    - RR — Round Robin (라운드 로빈)
    - SRT — Shortest Remaining Time (최소 잔여 시간 우선)
    - Preemptive Priority (선점 우선순위)
    - MLQ · MFQ — Multilevel Queue / Multilevel Feedback Queue (다단계 큐 / 다단계 피드백 큐)

### 3. 메모리를 나누고 교체하기

- **연속 배치 전략**
  - First Fit (최초 적합) · Best Fit (최적 적합) · Worst Fit (최악 적합)
- **불연속 할당**
  - Paging — 고정 크기 Page / Frame
  - Segmentation (세그멘테이션) — 논리적 가변 크기 Segment
- **필요한 페이지만 메모리에 적재**
  - Virtual Memory (가상 메모리)
    - Page Fault 발생 시 Page Replacement
      - FIFO (선입선출) · LRU (최근 최소 사용) · LFU (사용 빈도 최소) · NUR (최근 미사용) · OPT (최적 교체)
    - 페이지 부재가 지나치면 Thrashing (스래싱)
      - Working Set으로 완화

### 4. 네트워크를 계층으로 보기

- **OSI 7계층**
  - 7 Application — 사용자 서비스



- HTTP — HyperText Transfer Protocol (하이퍼텍스트 전송 프로토콜)
- FTP — File Transfer Protocol (파일 전송 프로토콜)
- SMTP — Simple Mail Transfer Protocol (간이 전자우편 전송 프로토콜)
- DNS — Domain Name System (도메인 이름 시스템)
- SNMP — Simple Network Management Protocol (간이 네트워크 관리 프로토콜)
- 6 Presentation — 형식 변환·압축·암호화
- 5 Session — 연결 세션 관리
- 4 Transport — 종단 간 전송
  - TCP — Transmission Control Protocol (전송 제어 프로토콜)
  - UDP — User Datagram Protocol (사용자 데이터그램 프로토콜)
  - 전송 단위 — Segment
- 3 Network — 주소와 경로
  - IP — Internet Protocol (인터넷 프로토콜)
  - ICMP — Internet Control Message Protocol (인터넷 제어 메시지 프로토콜)
  - IGMP — Internet Group Management Protocol (인터넷 그룹 관리 프로토콜)
  - ARP · RARP · RIP · OSPF · BGP
  - 장비 — Router
  - 전송 단위 — Packet
- 2 Data Link — 인접 노드 전송
  - Ethernet (이더넷) · MAC — Media Access Control Address (물리 주소)
  - LLC — Logical Link Control (논리적 링크 제어) · HDLC
  - 장비 — Bridge · Switch
  - 전송 단위 — Frame
- 1 Physical — 전기·물리 신호
  - 장비 — Repeater · Hub
  - 전송 단위 — Bit
- **IP 주소 체계**
  - IPv4 — 32bit (인터넷 프로토콜 버전 4)
    - Subnetting (서브네팅) · CIDR (무클래스 도메인 간 라우팅)
  - IPv6 — 128bit (인터넷 프로토콜 버전 6)
    - Unicast (유니캐스트) · Multicast (멀티캐스트) · Anycast (애니캐스트)
- **주소를 서로 변환**

- DNS — Domain Name System (도메인 이름 시스템): Domain Name → IP Address
- ARP — Address Resolution Protocol (주소 분석 프로토콜): IP Address → MAC Address
- RARP — Reverse Address Resolution Protocol (역주소 분석 프로토콜): MAC Address → IP Address
- **흐름 제어와 응답 신호**
  - Stop-and-Wait (정지-대기)
  - Sliding Window (슬라이딩 윈도우)
  - ACK — Acknowledgement (긍정 응답)
  - NAK — Negative Acknowledgement (부정 응답)
- **교재의 주요 프로토콜**
  - 데이터 링크·패킷망
    - Ethernet — IEEE 802.3
    - IEEE 802 LAN 표준
    - HDLC — High-Level Data Link Control (고급 데이터 링크 제어)
    - X.25 — DTE와 DCE 사이의 패킷 교환망 인터페이스
    - RS-232C — PSTN을 통한 DTE와 DCE 사이의 인터페이스
  - 실시간 전송
    - RTP — Real-time Transport Protocol (실시간 전송 프로토콜)
    - RTCP — Real-time Transport Control Protocol (실시간 전송 제어 프로토콜)
  - 원격 접속
    - TELNET — TELEcommunication NETwork (원격 접속 프로토콜)
  - 웹 전송
    - HTTP · HTTPS · S-HTTP
- **네트워크 연결 장치**
  - NIC — Network Interface Card (네트워크 인터페이스 카드)
- **전송 방식 선택**
  - TCP — 연결형·신뢰성
  - UDP — 비연결형·속도
- **경로를 선택하는 프로토콜**
  - 내부 라우팅
    - RIP — Routing Information Protocol: Distance Vector (거리 벡터)
    - OSPF — Open Shortest Path First: Link State (링크 상태)

- 외부 라우팅
    - EGP — Exterior Gateway Protocol (외부 게이트웨이 프로토콜)
    - BGP — Border Gateway Protocol: Path Vector (경로 벡터)
- 

## VI. 프로젝트와 인프라 관리

### 1. 개발 방법론을 선택하고 재사용하기

- 구조적 방법론
  - 정형화된 분석 절차와 자료 흐름 중심
- 정보공학 방법론
  - 기업의 정보 시스템 전체를 데이터 중심으로 구축
- CBD — Component Based Development (컴포넌트 기반 개발)
  - 재사용 가능한 컴포넌트를 조립해 시스템 구축
- 소프트웨어 재사용
  - Composition-Based (합성 중심)
  - Generation-Based (생성 중심)
- 소프트웨어 재공학
  - Analysis → Restructuring → Reverse Engineering → Migration
  - Reverse Engineering (역공학) ↔ Forward Engineering (순공학)
- 소프트웨어 개발 프레임워크
  - Modularity (모듈화) · Reusability (재사용성) · Extensibility (확장성)

### 2. 개발 규모와 비용을 추정

- 코드 양 기준
  - LOC — Lines of Code (원시 코드 라인 수)
- 기능 양 기준
  - FP — Function Point (기능 점수)
    - 입력 · 출력 · 질의 · 내부 파일 · 외부 인터페이스
- 경험식 기반 모델
  - COCOMO — Constructive Cost Model (구성적 비용 산정 모형)
    - Organic (조직형) · Semi-detached (반분리형) · Embedded Mode (내장형)
  - Putnam Model (푸트남 모형)
    - Rayleigh-Norden Curve (레일리-노든 곡선)

- 자동화 비용 산정 도구
  - SLIM
  - ESTIMACS

### 3. 프로젝트 작업과 일정을 계획

- 작업 순서와 기간을 계산
  - PERT — Program Evaluation and Review Technique (프로그램 평가 및 검토 기법)
    - 낙관 시간 · 최빈 시간 · 비관 시간으로 기대 시간 계산
  - CPM — Critical Path Method (임계 경로 기법)
    - ES (최조 시작) · EF (최조 종료) · LS (최지 시작) · LF (최지 종료) · Slack (여유 시간)
  - Gantt Chart — 시간축 일정 표시

### 4. 개발 프로세스의 성숙도를 관리

- 생명주기 표준
  - ISO/IEC 12207
- 조직 프로세스 성숙도
  - CMMI — Capability Maturity Model Integration (능력 성숙도 통합 모델)
  - SPICE — Software Process Improvement and Capability Determination (소프트웨어 프로세스 개선 및 능력 평가)
- 프로젝트에 맞게 방법론 조정
  - Tailoring (테일러링)

### 5. 소프트웨어로 인프라를 제어

- SDE / SDx — Software-Defined Everything (소프트웨어 정의 기술)
  - SDN — Software-Defined Networking (소프트웨어 정의 네트워킹)
    - 제어부와 전달부 분리
  - SDS — Software-Defined Storage (소프트웨어 정의 스토리지)
  - SDDC — Software-Defined Data Center (소프트웨어 정의 데이터센터)
- 컴퓨팅 위치에 따른 형태
  - Cloud Computing — 중앙 클라우드
- 네트워크 범위와 논리 분리
  - LAN (근거리 통신망) · WAN (광역 통신망)
  - VLAN — Virtual Local Area Network (가상 근거리 통신망)
  - Ad-hoc Network (애드혹 네트워크) · Mesh Network (메시 네트워크)

- 매체 접근 방식
  - 유선 Ethernet — CSMA/CD
  - 무선 LAN — CSMA/CA
  - 무선 LAN 표준
    - IEEE 802.11a — OFDM 사용
    - IEEE 802.11i — WPA / WPA2 인증

## 6. 교재의 주요 인프라·데이터 신기술

- 근거리·무선 통신
  - PICONET (피코넷)
  - UWB — Ultra-Wideband (초광대역)
  - NFC — Near Field Communication (근거리 무선 통신)
  - WDM — Wavelength Division Multiplexing (파장 분할 다중화)
- 하드웨어·저장 기술
  - HSM — Hardware Security Module (하드웨어 보안 모듈)
  - RAID — Redundant Array of Independent Disks (독립 디스크 중복 배열)
  - MEMS — Micro-Electro-Mechanical Systems (초소형 정밀기계 기술)
  - RISC — Reduced Instruction Set Computer (축소 명령어 집합 컴퓨터)
  - ARM — Advanced RISC Machine (고급 RISC 머신)
- 서비스·데이터 처리 구조
  - SOA — Service-Oriented Architecture (서비스 지향 아키텍처)
  - OLAP — Online Analytical Processing (온라인 분석 처리)
    - Roll-up · Drill-down · Drill-through · Drill-across
    - Pivoting · Slicing · Dicing
  - DPI — Deep Packet Inspection (심층 패킷 검사)

## VII. 시스템을 안전하게 지키기

### 1. 보안을 적용하는 위치

- 관리적 보안
  - 정책 · 절차 · 교육 · 인적 통제
- 물리적 보안
  - 출입 통제 · 시설 · 장비 보호

- **기술적 보안**
  - 인증 · 접근통제 · 암호화 · 보안 시스템
- **Secure OS의 보호 대상과 기능**
  - 보호 대상
    - 메모리 · 보조기억장치의 파일 · 프로세스 · 보호 가능한 공유 자원
  - 보안 기능
    - 사용자 식별 및 인증
    - 임의적·강제적 접근통제
    - 객체 재사용 보호
    - 완전한 조정
    - 신뢰 경로
    - 감사 및 감사기록 축소
- **Secure SDLC (보안 소프트웨어 개발 생명주기)**
  - 요구사항 — Security Requirements
  - 설계 — Threat Modeling
  - 구현 — Secure Coding
  - 테스트 — Security Testing
  - 운영 — Patch · Monitoring · Incident Response
  - 관련 방법론
    - SDL — Security Development Lifecycle (보안 개발 생명주기)
    - CLASP — Comprehensive, Lightweight Application Security Process (종합적 경량 애플리케이션 보안 프로세스)

## 2. 신원을 확인하고 권한을 부여

- **인증과 권한 부여**
  - Authentication (인증) — 누구인가
  - Authorization (인가) — 무엇을 할 수 있는가
- **인증 수단**
  - 지식 — Password
  - 소유 — Token · Smart Card
  - 생체 — Fingerprint · Iris · Face
  - 통합 인증
    - SSO — Single Sign-On (통합 인증)

- 접근통제 방식
  - DAC — Discretionary Access Control (임의적 접근통제)
    - 소유자가 권한 결정
  - MAC — Mandatory Access Control (강제적 접근통제)
    - 보안 등급이 권한 결정
    - Bell-LaPadula — 기밀성
    - Biba — 무결성
  - RBAC — Role-Based Access Control (역할 기반 접근통제)
    - 역할이 권한 결정

### 3. 암호 체계와 알고리즘

- 복호화할 수 있는 암호화
  - 대칭키 암호화 — 암호화 키와 복호화 키가 같음
    - 블록 암호
      - DES — Data Encryption Standard (데이터 암호화 표준)
      - 3DES — Triple DES (삼중 DES)
      - AES — Advanced Encryption Standard (고급 암호화 표준)
      - SEED (시드)
      - ARIA (아리아)
      - LUCIFER (루시퍼)
    - 스트림 암호
      - RC4 — Rivest Cipher 4 (리베스트 암호 4)
      - LFSR — Linear Feedback Shift Register 계열
  - 비대칭키 방식 — 공개키와 개인키를 사용
    - 암호화
      - RSA — Rivest, Shamir, Adleman (RSA 암호)
      - ECC — Elliptic Curve Cryptography (타원곡선 암호) 기반 암호
      - Rabin (라빈)
- 복호화하지 않는 단방향 변환
  - Hash Function (해시 함수)
    - MD4 — Message-Digest algorithm 4 (메시지 다이제스트 알고리즘 4)
    - MD5 — Message-Digest algorithm 5 (메시지 다이제스트 알고리즘 5)

- SHA — Secure Hash Algorithm (안전한 해시 알고리즘)
  - SHA-0 · SHA-1
  - SHA-2 — SHA-224 · SHA-256 · SHA-384 · SHA-512
- N-NASH (엔-내시)
- SNEFRU (스네프루)
- 비밀번호 저장 보강
  - Salt (솔트)
- 암호 기술로 만드는 보안 기능
  - 기밀성
    - 대칭키·비대칭키 암호화
  - 무결성
    - Hash (해시)

## 4. 보안 통신 프로토콜

- 암호화된 통신 경로
  - SSL — Secure Sockets Layer (보안 소켓 계층)
    - HTTPS에 사용
  - SSH — Secure Shell (보안 셸)
    - 원격 접속 보호
  - IPSec — Internet Protocol Security (인터넷 프로토콜 보안)
    - IP 계층 보호
  - VPN — Virtual Private Network (가상 사설망)
    - 공용망 위에 암호화 터널 구성

## 5. 취약점은 발생 위치로 구분

- 입력값 검증 실패
  - SQL Injection — 입력이 SQL 명령으로 실행
  - XSS — Cross-Site Scripting (교차 사이트 스크립팅)
  - Command Injection (명령어 삽입)
  - Path / Directory Traversal (경로 조작)
- 보안 약점 확인 기준
  - OWASP — Open Worldwide Application Security Project (오픈 웹 애플리케이션 보안 프로젝트)
  - OWASP Top 10



- 웹 요청·세션 신뢰 실패
  - Session Hijacking (세션 하이재킹)
  - Cookie 변조
- 메모리와 자료형 처리 실패
  - Buffer Overflow (버퍼 오버플로)
    - Stack Guard (스택 가드)로 완화
  - Integer Overflow (정수 오버플로)
  - Format String Attack (포맷 스트링 공격)
- 권한 설계 실패
  - 접근 지정자
    - Public · Protected · Default · Private

## 6. 네트워크 공격은 목적에 따라 구분

- 서비스를 못 쓰게 함 — DoS / DDoS
  - 자원 고갈
    - SYN Flooding (SYN 플러딩)
  - 반사·증폭
    - Smurfing (스머핑)
  - 비정상 패킷
    - Ping of Death (죽음의 핑) · Teardrop (티어드롭) · Land Attack (랜드 공격)
    - Ping Flood (핑 플러드)
  - DDoS 공격용 도구
    - Trin00
    - TFN — Tribe Flooding Network (트라이브 플러딩 네트워크)
    - TFN2K
    - Stacheldraht
- 통신을 엿보거나 중간에서 조작
  - Sniffing (스니핑)
  - Session Hijacking (세션 하이재킹)
- 다른 대상을 사칭
  - IP Spoofing (IP 스푸핑)
  - ARP Spoofing (ARP 스푸핑)

- DNS Spoofing (DNS 스푸핑)
- **사람을 속여 정보를 획득**
  - Social Engineering (사회공학)
    - Phishing (피싱) · Pharming (파밍) · Smishing (스미싱)
    - Shoulder Surfing (숄더 서핑) · Dumpster Diving (덤스터 다이빙)
- **무선 LAN·스위치 대상 공격**
  - Evil Twin Attack (이블 트윈 공격)
  - Switch Jamming (스위치 재밍)
- **Bluetooth 대상 공격**
  - BlueJacking (블루재킹) · BlueSnarf (블루스나프)
  - BlueBug (블루버그) · BluePrinting (블루프린팅)

## 7. 공격을 탐지하고 차단하는 시스템

- **네트워크 경계 통제**
  - Firewall (방화벽)
  - DMZ — Demilitarized Zone (비무장지대)
- **침입 탐지와 차단**
  - IDS — Intrusion Detection System (침입 탐지 시스템)
    - 탐지 후 알림
  - IPS — Intrusion Prevention System (침입 방지 시스템)
    - 탐지 후 차단
  - 탐지 방식
    - Signature-based — 오용 탐지
    - Anomaly-based — 이상 탐지
  - 설치 위치에 따른 IDS
    - HIDS — Host-Based Intrusion Detection System (호스트 기반 침입 탐지 시스템)
      - OSSEC · md5deep · AIDE · Samhain
    - NIDS — Network-Based Intrusion Detection System (네트워크 기반 침입 탐지 시스템)
      - Snort · Zeek
- **공격자를 유인해 관찰**
  - Honeypot (허니팟)

## 8. 보안 로그

- **Linux에서 확인하는 주요 로그**

- `/var/log/messages` — 일반 시스템 메시지
  - `/var/log/secure` — 인증·보안
  - `/var/log/wtmp` — 성공한 로그인
  - `/var/log/btmp` — 실패한 로그인
  - `/var/log/lastlog` — 사용자별 최근 로그인
- 

## 마지막 연결 지도

- 요구사항은 **UML·UI·아키텍처**의 입력이 된다.
- 아키텍처는 **모듈과 인터페이스의 경계**를 정한다.
- 모듈 내부에서는 **자료구조와 알고리즘**이 동작한다.
- 프로그램은 **운영체제의 프로세스와 메모리** 위에서 실행된다.
- 프로그램 사이의 통신은 **네트워크 프로토콜과 EAI·ESB·웹 서비스**가 담당한다.
- 영속 데이터는 **DBMS와 SQL**이 관리한다.
- 형상 관리·빌드·테스트를 통과한 결과물이 **패키징되어 배포**된다.
- 보안은 마지막 단계가 아니라 **요구사항부터 운영까지 전 과정**에 적용된다.